TJM LABS · TAKE-HOME

Design Doc: Inbound Pharmacy Sales Agent

Muhammad Tameem Rafay · rafaytariq369616@gmail.com

1. Context & Problem

Pharmacies call a phone number listed on the TJM Labs website. We want an AI-powered inbound sales agent that can:

1. **Identify** who is calling (from their phone number),
2. **Understand** their intent, and
3. **Act** — pitch TJM Labs' value (especially to high-Rx-volume pharmacies) and arrange a concrete follow-up (email or callback).

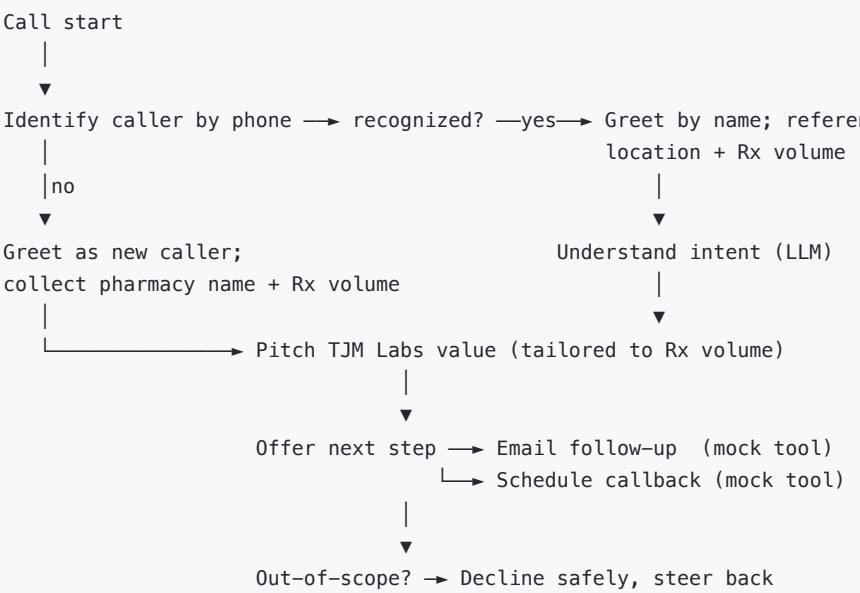
This document describes how I'd build the full system, then how the included implementation realizes a simplified, text-based slice of it.

2. Assumptions

Because the brief is intentionally open-ended, I made these explicit choices:

- **Caller ID is available.** A real telephony layer (Twilio/Vonage) provides the caller's number on connect. Here it's a mock variable (`--phone`).
- **"Rx volume" is derived.** The directory API returns a `prescriptions: [{drug, count}]` array, not a single volume figure — so total monthly Rx volume = **sum of counts**. "High volume" is a configurable threshold (default **100/mo**).
- **One pharmacy per phone number.** First exact match wins. Real data may need fuzzy/multi-location handling.
- **Text simulates voice.** The turn-by-turn loop stands in for STT/TTS. Replies are kept short and "phone-like."
- **Follow-up actions are mocked.** Email/callback are side-effect stubs that log and print; wiring real providers is out of scope.
- **Safety over completeness.** When unsure or off-topic, the agent declines and offers a follow-up rather than guessing. No hallucinated facts.

3. Conversation Flow



Two headline behaviors:

- **Recognized:** "Hi HealthFirst Pharmacy, thanks for calling! Great to hear from you in New York, NY — I see you're filling ~100 Rx/month…"
- **Unrecognized:** collect name → collect Rx volume → tailored pitch, using the pharmacy name throughout once known.

4. System Design

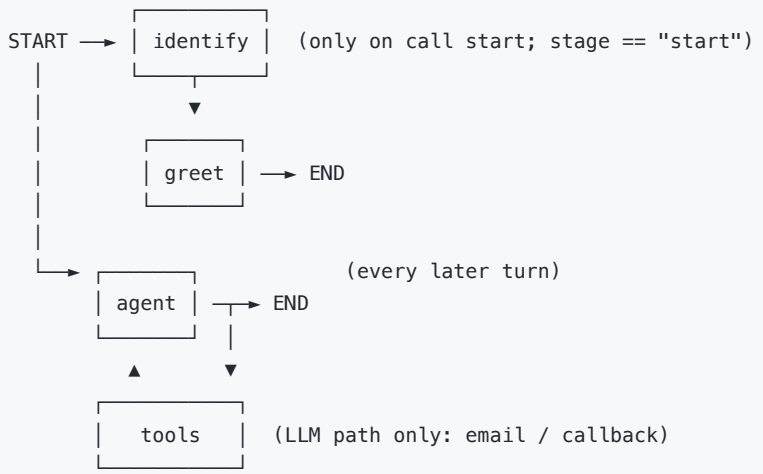
4.1 Architecture (implemented)

A small, layered Python package built on **LangGraph**:

```
main.py              # entry point (python main.py)
pharmacy_agent/
├── config.py         # env-driven settings (frozen dataclass)
├── logging_config.py # consistent, timestamped logging
├── models.py         # Pharmacy / Prescription; derives Rx volume
├── pharmacy_api.py    # directory API client + phone identification
├── llm.py            # Claude accessor + availability check
├── prompts.py        # grounded system prompt + TJM value prop
├── tools.py          # mocked email / callback (LangChain @tool)
├── fallback.py       # deterministic no-LLM conversation engine
├── state.py          # LangGraph state (TypedDict)
├── graph.py          # StateGraph: identify → greet → agent ⇄ tools
└── cli.py            # text REPL simulating a call
```

4.2 The graph

State is threaded through nodes; `messages` uses the `add_messages` reducer, and a `MemorySaver` checkpointer (keyed by a `thread_id`) preserves state so the CLI can drive **one turn per invocation**.



- **identify** — calls the directory API, matches the caller by normalized phone digits, and builds a *grounded* system prompt from verified data.
- **greet** — opening line (LLM when available, else a factual template).
- **agent** — one conversational turn. LLM path binds the follow-up tools and uses `tools_condition` to route into a `ToolNode`; fallback path runs the deterministic engine.

4.3 Identification

Phone numbers are normalized to bare digits (drop `+1`, spaces, dashes) before comparison, so `+1-555-123-4567`, `(555) 123-4567`, and `15551234567` all match. A directory outage is caught and degrades to the "unrecognized caller" flow instead of crashing the call.

4.4 Observability

Structured, leveled logging throughout (identification result, greeting, tool invocations, API errors). Mock tools both `log` and `print` a clear confirmation block so the side effect is visible in the transcript.

5. LLM Strategy

- **Cache-friendly, XML-structured prompt (implemented).** The system prompt is split into a *static* section (role, product facts, rules, tool guidance) and a *dynamic* `<caller_context>` section. The static part is placed **first** and carries an Anthropic

`cache_control` breakpoint, so Claude caches this prefix and reuses it on every call — only the small per-call caller context is re-processed, cutting latency and token cost. All sections use XML tags (`<role>` , `<about_tjm_labs>` , `<rules>` , `<tools>` , `<caller_context>`) because Claude follows XML-delimited structure more reliably, and it lets us point the model at `<caller_context>` as its single source of truth.

- **Grounding, not memorization.** Every fact the agent may state about the caller is injected into the system prompt from verified API data. The model is explicitly instructed to state only what it's given or told — this is the primary anti-hallucination control.
- **Guardrails in the prompt.** Rules cover scope (pharmacy sales / TJM Labs only), "say you don't know rather than guess," and "only call a tool after the caller agrees."
- **Tools as the action surface.** `send_email_followup` and `schedule_callback` are bound to the model; the LLM decides *when*, LangGraph executes *how*.
- **Deterministic greeting option.** Factual greetings can be templated to guarantee correctness; conversation stays LLM-driven for flexibility.
- **Graceful degradation.** With no `ANTHROPIC_API_KEY` , a rule-based engine takes over. Because its replies are fully templated, it *cannot* hallucinate — a useful safety floor and a way to demo the system anywhere.
- **Model choice.** Defaults to `claude-sonnet-5` ; low temperature (0.3) for consistent, on-brand sales copy. Swappable via env.

6. Tradeoffs

Decision	Why	Cost / risk
LangGraph state machine	Explicit, inspectable control flow; easy to add nodes (CRM, escalation)	More scaffolding than a single prompt loop
Derive Rx volume from counts	API has no volume field	Threshold (100) is a guess; should be data-driven
Grounded prompt + templated fallback	Strong anti-hallucination posture	Fallback is intentionally rigid / less natural
One turn per graph invocation + checkpointer	Clean REPL, resumable, testable	In-memory checkpointer isn't durable across restarts
Fetch whole directory, match locally	Simple, provider-agnostic	Won't scale to large directories (see §7)
Mocked follow-up tools	In scope for the task	No real delivery/idempotency yet

7. Written Question — "If I had 3 more hours"

First, I'd build an evaluation pipeline. A sales agent is only trustworthy if we can measure it, so before adding features I'd invest in evaluation:

- A **dataset of scripted call scenarios** (recognized/unrecognized, high/low volume, off-topic probes, tool-request phrasings, adversarial "get it to hallucinate" prompts), each with expected behaviors.
- An **automated harness** that runs each scenario through the graph and scores it on: correct identification, greeting-by-name, factual grounding (no invented details), scope adherence, and whether the right tool fired with the right args.
- An **LLM-as-judge** rubric for the fuzzy dimensions (tone, helpfulness, did-it-stay-on-script), plus regression tracking so prompt changes are measured, not guessed. This makes every later change safe to ship.

Then, with the remaining time:

- **Distributed tracing (LangSmith).** Instrument the whole graph with LangSmith so every production call is traced end-to-end — node inputs/outputs, prompts, tool calls, token usage and latency. If something goes wrong on a live call we can trace it back to the exact node and reproduce it, instead of guessing.
- **Redis caching.** Cache the pharmacy directory / caller lookups and the results of *read-only, deterministic* tool calls in Redis (with TTLs), so repeat access is served from cache instead of hitting the directory API/DB on every call. (Only cache safe-to-cache reads — never side-effecting actions like actually sending an email or booking a callback.)
- **Intent classification node** — an explicit intent step (new lead, existing customer support, pricing, billing, wrong number) to branch the flow and route to the right playbook or a human.
- **Cross-call memory (conversation summaries).** At the end of each call, run a summarization pass and persist a short summary keyed by the caller / pharmacy. On their next call, load that prior summary into the prompt (as another cache-friendly

context block) so the agent remembers past conversations and picks up where it left off — better continuity and a more personal relationship than starting cold every time.

- **Real tool integrations** — wire `send_email_followup` to an email provider and `schedule_callback` to a calendar/CRM (e.g. Salesforce/HubSpot), with idempotency and delivery logging.
- **Scalable identification** — query the directory by phone server-side (`?phone=`) or cache it, instead of fetching everything; handle multi-location and fuzzy matches.
- **Robustness & persistence** — retries/timeouts on the API, a durable checkpointer (Redis/Postgres) so calls survive restarts, and full transcript persistence for audit.
- **Horizontal scaling & call routing (stateless + shared state).** When servers scale out, a call must reach an instance that has its state. The simple, production-ready approach is to keep instances **stateless**: externalize agent state to the shared LangGraph checkpointer (Redis/Postgres) keyed by the telephony call id (e.g. Twilio `CallSid`), and put instances behind a load balancer. Any instance can then serve any turn by loading state from the shared store, and a crashed instance's call can resume elsewhere. For the real-time media stream, the provider holds one long-lived WebSocket to a single instance for the call's duration (natural per-call affinity), backed by the shared checkpoint for failover. *References: Twilio Media Streams (`twilio.com/docs/voice/media-streams`); LangGraph persistence / checkpointers (`langchain-ai.github.io/langgraph/concepts/persistence`); load-balancer session affinity — AWS ALB sticky sessions / consistent hashing.*
- **Voice + metrics** — an STT/TTS adapter behind the same graph, plus product metrics (conversion rate, tool-usage, handoff rate).
- **Guardrail hardening** — a dedicated safety/scope check and a "confidence → human handoff" path for anything the agent shouldn't answer.